



Universidad Autónoma de Chihuahua
Facultad de Ingeniería



Ingeniería en ciencias de la computación
Cómputo Paralelo y Distribuido

SISTEMA DISTRIBUIDO PARA ANÁLISIS DE SINIESTROS VIALES CON DATOS ABIERTOS

REPORTE TÉCNICO — PROYECTO FINAL

Manuel Martinez Martinez 368064
Rebeca Portillo Saenzpardo 368094
Pablo Gael Torre 368073
Fernando Grijalva Fernández 367634

Fecha: 26 de mayo de 2026

1. Descripción General

El presente proyecto implementa una plataforma de análisis distribuido de accidentes viales en México, utilizando datos abiertos del INEGI (ATUS — Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas). La aplicación procesa registros históricos de siniestros para identificar patrones de riesgo por entidad federativa, municipio, hora, mes, causa y gravedad.

Se emplean las bibliotecas Ray y Ray Cluster de Python para distribuir el procesamiento en múltiples nodos y/o núcleos, logrando una reducción significativa en los tiempos de análisis frente al enfoque secuencial con Pandas.

2. Problema a Resolver

Cuando se acumulan datos históricos de varios años, el procesamiento secuencial de grandes volúmenes de accidentes viales se vuelve lento e ineficiente. Las instituciones que requieren este análisis incluyen:

- Municipios y dependencias de movilidad urbana
- Escuelas de manejo y capacitación vial
- Compañías aseguradoras
- Instituciones académicas y de investigación

El sistema responde las siguientes preguntas analíticas:

Pregunta de análisis	Resultado esperado
¿Qué estados concentran más accidentes?	Ranking por entidad federativa
¿Qué municipios presentan mayor siniestralidad?	Tabla comparativa por municipio
¿En qué horarios ocurren más accidentes?	Distribución por hora del día
¿Qué causas son más frecuentes?	Ranking de causas
¿Qué meses presentan mayor incidencia?	Tendencia mensual
¿Dónde hay más víctimas heridas o fallecidas?	Análisis por zona y gravedad
¿Qué tan eficiente es Ray vs. Pandas?	Comparación tiempos y speedup

3. Arquitectura del Sistema

El sistema sigue una arquitectura de cuatro capas:

Componente	Función principal
Fuente ATUS (INEGI)	Registros históricos de accidentes viales en CSV/Parquet
Módulo de limpieza	Estandariza columnas, tipos de datos, fechas, horas y valores faltantes
Ray Head Node	Coordina el cluster, administra tareas, objetos y workers

Ray Worker Nodes	Ejecutan tareas distribuidas sobre particiones de datos por estado
Dashboard Streamlit	Rankings, gráficas, indicadores y comparación de rendimiento interactiva

3.1 Flujo de datos

- Los datos ATUS se cargan desde CSV o Parquet
- El módulo de limpieza normaliza y valida registros
- Ray pone el dataset en el Object Store (ray.put)
- El dataset se particiona en N bloques (N = número de CPUs/workers)
- Tareas Ray (@ray.remote) procesan particiones en paralelo
- Una fase de reducción combina los resultados parciales
- Los resultados finales se exportan a CSV y se visualizan en Streamlit

4. Implementación

4.1 Estructura del proyecto

Archivo	Descripción
src/generar_datos.py	Genera dataset sintético con estructura ATUS-INEGI (500K-2M registros)
src/limpieza.py	Limpieza, normalización y exportación a Parquet
src/analisis_secuencial.py	Análisis con Pandas puro — línea base para benchmark
src/analisis_ray.py	Análisis distribuido con Ray: tareas @ray.remote + reducción
src/benchmark.py	Compara tiempos y calcula speedup entre los tres modos
src/dashboard.py	Dashboard interactivo con Streamlit y Plotly
src/main.py	Orquestador principal — ejecuta todo el pipeline
docker/Dockerfile	Imagen Docker con Python, Ray y Streamlit
docker/docker-compose.yml	Cluster completo: 1 Head Node + 3 Worker Nodes + App + Dashboard

4.2 Procesos distribuidos con Ray

Se implementaron las siguientes tareas remotas (@ray.remote):

Tarea Ray	Paralelización	Reducción
tarea_por_estado()	Por bloques de registros	Suma y agrupa por estado

tarea_por_municipio()	Por bloques de registros	Suma y agrupa por municipio
tarea_por_hora()	Por bloques	Suma por hora (0–23)
tarea_por_causa()	Por bloques	Ranking de causas
tarea_por_mes()	Por bloques	Tendencia mensual
tarea_resumen_general()	Dataset completo (ray.put)	Métricas globales

5. Resultados e Indicadores

El sistema genera los siguientes indicadores de siniestralidad:

Indicador	Descripción
Total de accidentes	Número total de registros analizados
Accidentes con heridos	Accidentes donde hubo personas lesionadas
Accidentes con fallecidos	Accidentes con víctimas mortales
Distribución por hora	Frecuencia de accidentes por hora del día (0–23 hrs)
Ranking por municipio	Top 50 municipios con más siniestros
Ranking de causas	Causas más frecuentes ordenadas por incidencia
Índice de gravedad	$(\text{heridos} + \text{fallecidos} \times 3) / \text{accidentes por estado}$
Tendencia anual	Evolución histórica de accidentes, heridos y fallecidos
Speedup Ray vs. Pandas	Ratio de aceleración en tiempo de procesamiento

6. Instrucciones de Ejecución

6.1 Ejecución local (modo paralelo — 80%)

Requisitos: Python 3.11+, pip

1. Instalar dependencias

```
pip install -r requirements.txt
```

2. Ejecutar pipeline completo con benchmark

```
python src/main.py --benchmark
```

3. Lanzar dashboard

```
streamlit run src/dashboard.py
```

6.2 Ejecución con Ray Cluster en Docker (modo distribuido — 100%)

Requisitos: Docker Desktop o Docker Engine + Docker Compose

1. Levantar el cluster completo

```
cd docker && docker compose up --build
```

2. Escalar a más workers si se desea

`docker compose up --scale worker=5`

3. Acceder a los dashboards:

- Ray Dashboard: `http://localhost:8265`
- Streamlit Dashboard: `http://localhost:8501`

4. Bajar el cluster

`docker compose down`

6.3 Usar datos reales del INEGI

- Descargar el CSV de ATUS en: <https://www.inegi.org.mx/programas/accidentes/>
- Colocar el archivo en la carpeta `data/`
- Ejecutar: `python src/main.py --datos data/tu_archivo.csv`

7. Conclusiones

El sistema implementado demuestra la viabilidad del procesamiento distribuido con Ray para el análisis de grandes volúmenes de datos de siniestralidad vial. Las principales conclusiones son:

- Ray permite distribuir el análisis de forma transparente entre CPUs locales y nodos remotos del cluster, sin cambios en la lógica de negocio.
- El speedup obtenido escala aproximadamente de forma lineal con el número de CPUs disponibles, confirmando la eficiencia del modelo de tareas remotas.
- La fase de reducción (Map-Reduce) es el cuello de botella principal; se optimizó usando `ray.put()` para evitar re-serIALIZACIÓN del dataset.
- El dashboard Streamlit + Plotly proporciona una interfaz accesible para usuarios no técnicos, permitiendo filtrar por estado, año o causa de forma interactiva.
- La arquitectura con Docker Compose facilita el despliegue reproducible del cluster en cualquier entorno con Docker disponible.

El proyecto cumple con todos los objetivos planteados y puede extenderse fácilmente para incorporar datos en tiempo real, modelos predictivos o integración con GIS para análisis geoespacial.